Programación funcional en Python: cómo he aplicado el paradigma

En todo el código intento lo mismo: funciones puras (mismo argumento, mismo resultado, sin variables externas ni modificar lo que reciben), sin bucles `for` ni `while`, y trabajando con `map`, `filter`, `reduce`, lambdas, closures y funciones que reciben o devuelven otras funciones. Cada función tiene al menos cinco llamadas de comprobación al final del script.

Parte 1 (parte1_funciones.py)

es_palindromo. Normalizo el texto encadenando `filter`: primero descarto los caracteres combinantes tras descomponer en NFD (así desaparecen las tildes) y luego me quedo solo con letras y dígitos con `str.isalnum`. Después comparo la cadena con su inversa. No modifico ninguna variable; cada paso produce un valor nuevo.

palindromos. Es un `map` de `es_palindromo` sobre la lista de entrada, convertido a lista. Paso la función como argumento en lugar de escribir un bucle.

cuadrados_sumados. `reduce` con una lambda que acumula *k·k* sobre `range(1, n+1)`, con 0 como valor inicial (por eso *n* = 0 da 0).

factorial. También con `reduce` y una lambda que multiplica. Evito la recursión, que se queda sin pila con números grandes, y el valor inicial 1 resuelve el caso de 0! y 1!.

filtrar_aprobados. `filter` con un predicado que fabrica la función `promedio_al_menos(minimo)`. Esa función devuelve una lambda que recuerda el valor de *minimo* (closure), de modo que la nota de corte (6) es un parámetro y no está escrita dentro del predicado. La lista original no se modifica; devuelvo una nueva.

Parte 2 (parte2_haveibeenpwned.py)

Callbacks para elegir el hash. El diccionario `FORMATOS` asocia la opción del usuario (1 o 2) con una tupla que contiene la propia función de hash (`hash_sha1` o `hash_ntlm`) y los parámetros de la petición (`mode=ntlm` en el segundo caso). `comprobar_contrasena` recibe ese formato y llama a `formato.hashear(...)` sin saber de cuál se trata. Añadir otro formato sería añadir una entrada a la tabla. NTLM es el MD4 de la contraseña en UTF-16 little endian; como `hashlib` ya no suele traer MD4, uso *pycryptodome*.

Callbacks para cada caso de la respuesta. `gestionar_respuesta` busca el código HTTP en una tabla de manejadores (200, 400, 403, 404, 429, 503) y ejecuta el que corresponda; si el código no está, aplica uno por defecto. Los manejadores se fabrican con funciones de orden superior: `error(mensaje)` devuelve un callback, y `partial(contar_filtraciones, sufijo)` fija el sufijo buscado. Para los rangos, `contar_filtraciones` procesa las líneas *SUFIJO:CUENTA* con `filter`, `map` y `reduce` (el 0 de las entradas de relleno no suma). Con dominios, un 404 se interpreta como «sin filtraciones», no como error, y las brechas se resumen con `map` y se ordenan por fecha.

Errores de conexión. El decorador `protegida` envuelve las consultas y traduce las excepciones de *requests* (tiempo agotado, sin conexión, JSON ilegible) a un resultado de error mediante una tabla de mensajes, de modo que el resto del código nunca ve una excepción.

Callbacks para el modo. `MODOS` asocia «1» con `modo_contrasenas` y «2» con `modo_dominios`; `ejecutar` elige el callback con `despachar` (el mismo mecanismo se usa para el hash y para los códigos HTTP) y cae en `modo_invalido` si la opción no existe.

Separar lo puro de los efectos. Las funciones no llaman a `input`, `print` ni `requests.get` directamente: los reciben en un `Entorno` (namedtuple). Los modos devuelven el mensaje y solo `ejecutar` lo muestra. Gracias a eso los ejemplos corren sin red, con respuestas simuladas (`python parte2_haveibeenpwned.py --ejemplos`). La contraseña completa no sale del equipo: solo se envían los cinco primeros caracteres del hash, y se pide con `getpass` para que no se vea al escribir.

Cómo probarlo

`pip install -r requirements.txt`. Después `python parte1_funciones.py` ejecuta los ejemplos de la parte 1, `python parte2_haveibeenpwned.py --ejemplos` los de la parte 2 sin red, y sin argumentos arranca el programa interactivo contra la API real. Las marcas y brechas de los ejemplos con respuestas simuladas son inventadas.